Viscan Dental Assistant: Verification Walkthrough

Architecture: Decoupled LLM Parser + Python Domain Validator	**Test Suite:** 37 passed (pytest tests/)
Live URL: https://viscan-dental-assistant.onrender.com	**Model:** nvidia/nemotron-3.5-lightning:free via OpenRouter
Mock Data: 6 synthetic patients, 13 synthetic reports in JSON	**Repo:** github.com/surendra902/viscan-dental-assistant

Architecture & Specification Alignment:

The brief asks for LLM interpretation to be separate from business-rule validation, so I split the system along that line. The LLM (OpenRouter Nemotron) only proposes candidate findings from dictated text: an anatomical site and an observation. Everything that decides what counts as fact is deterministic Python. find_patient and get_latest_report read the mock JSON data, and validators.py checks each candidate against a dental anatomy and pathology allowlist before it reaches the draft. Report retrieval and follow-ups use a rule-based parser with the same tools, so those answers come from stored report text. If the LLM call fails or times out, a local parser takes over and the same validation applies. The UI has two panels: the left handles report lookup and multi-turn follow-ups (Features A and C), and the right handles dictation, editable JSON and draft review (Feature B).

Known Limitations & Scope:

- **Synthetic dataset only:** Contains 6 mock patients and 13 reports. Not tested against hospital PACS/RIS databases.
- **Not clinically validated:** Built as an engineering prototype for course evaluation, not for actual diagnostic use.
- **Regex fallback edge cases:** Entity extraction relies on regex and fuzzy matching; unusual grammar or sentence structures can miss slots.
- **Free-tier model latency:** OpenRouter free tier can experience latency spikes; the system falls back to local rules if the API times out.

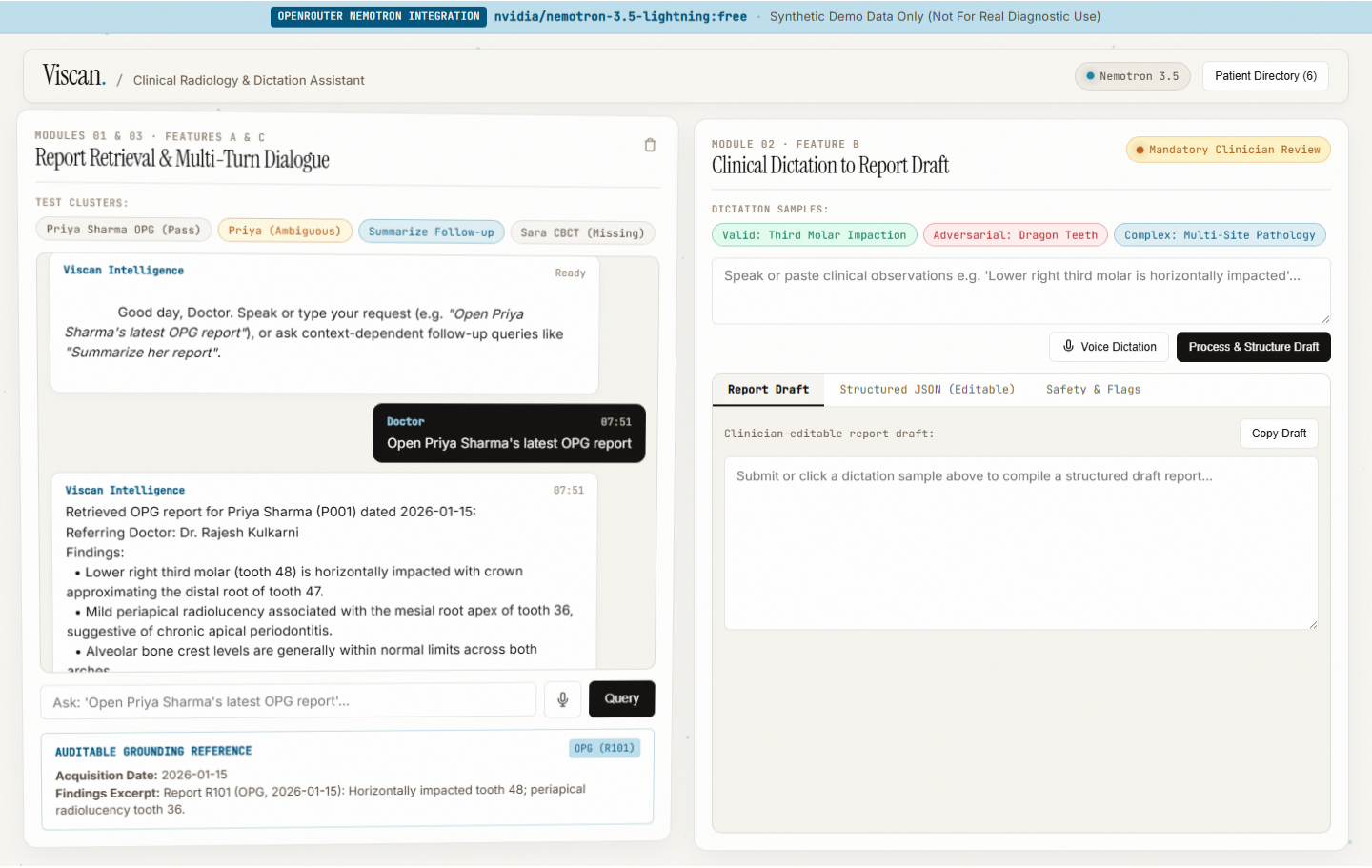
#	Feature	Input Prompt / Action	What the System Does
01	Feature A	Open Priya Sharma's latest OPG report	Extracts name and modality; pulls report R101 with grounding snippet.
02	Feature C	Summarize her report	Resolves pronoun 'her' to P001 from in-memory session history.
03	Feature C	when did she came to hospital	Temporal query over active patient records; returns 3 visit dates.
04	Feature C	what about her CBCT?	Modality switch for active patient; pulls 3D scan R103.
05	Feature A	Open Priya's latest OPG report	Detects two matches (P001, P002); halts and asks doctor to clarify.
06	Feature A	Open Sara Khan's latest CBCT report	Sara Khan has an IOPA but no CBCT; returns not_found without guessing.
07	Feature B	Lower right third molar is horizontally impacted	Validates site and observation against dental dictionary; creates draft.
08	Feature B	Structured JSON Schema View	Allows doctor to edit parsed JSON directly before committing.
09	Feature B	Patient has dragon teeth with quantum decay	Rejects non-dental terms; sets confidence to 0.30 and adds warning banner.
10	Safety	show all patients & Arjun Patil typo	Blocks bulk data dump; offers fuzzy suggestion for misspelled name.

Scenario 01: Exact Report Retrieval (Feature A)

FEATURE A · REPORT RETRIEVAL

Query / Input: "Open Priya Sharma's latest OPG report"

Input: Doctor asks for Priya Sharma's latest OPG scan.
Execution: The regex parser extracts patient name 'Priya Sharma' and modality 'OPG'. The function find_patient() returns ID P001. get_latest_report(P001, 'OPG') pulls record R101 (2026-01-15), which details an impacted tooth 48 and periapical lesion on tooth 36. The UI displays the text response and renders the bottom evidence card linking directly to R101.



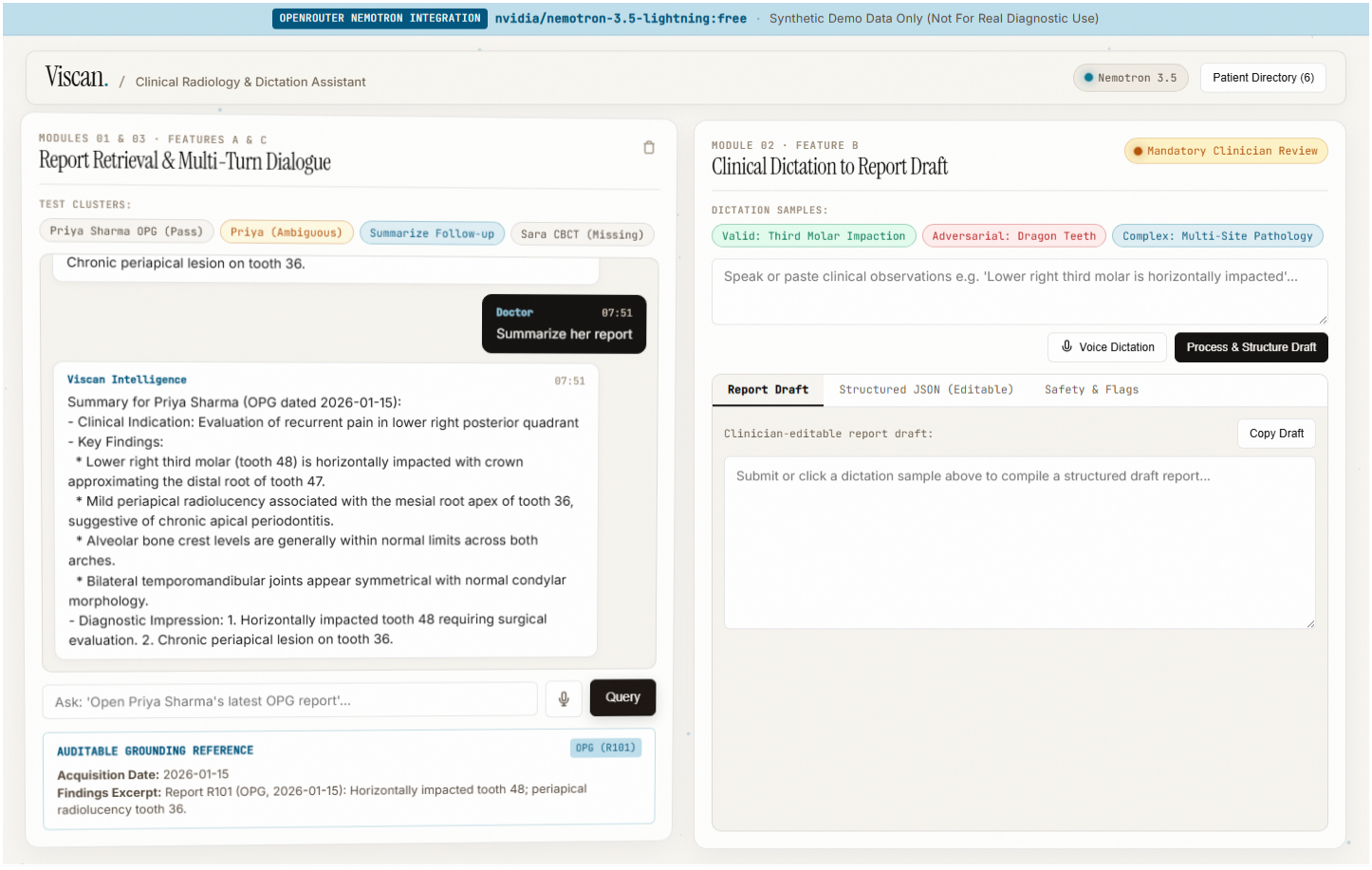
Verification: Response: HTTP 200 | JSON status: success | Matched: Priya Sharma (P001) -> Report R101

Scenario 02: Multi-Turn Pronoun Context (Feature C)

FEATURE C · PRONOUN RESOLUTION

Query / Input: "Summarize her report"

Input: Immediately following Scenario 1, doctor asks to summarize 'her' report.
Execution: Because no name is supplied, the router checks session.last_patient_id (P001) and session.last_report_id (R101). It builds a 3-part summary (Indication, Key Findings, Impression) from the active report without prompting the user to re-enter patient details.



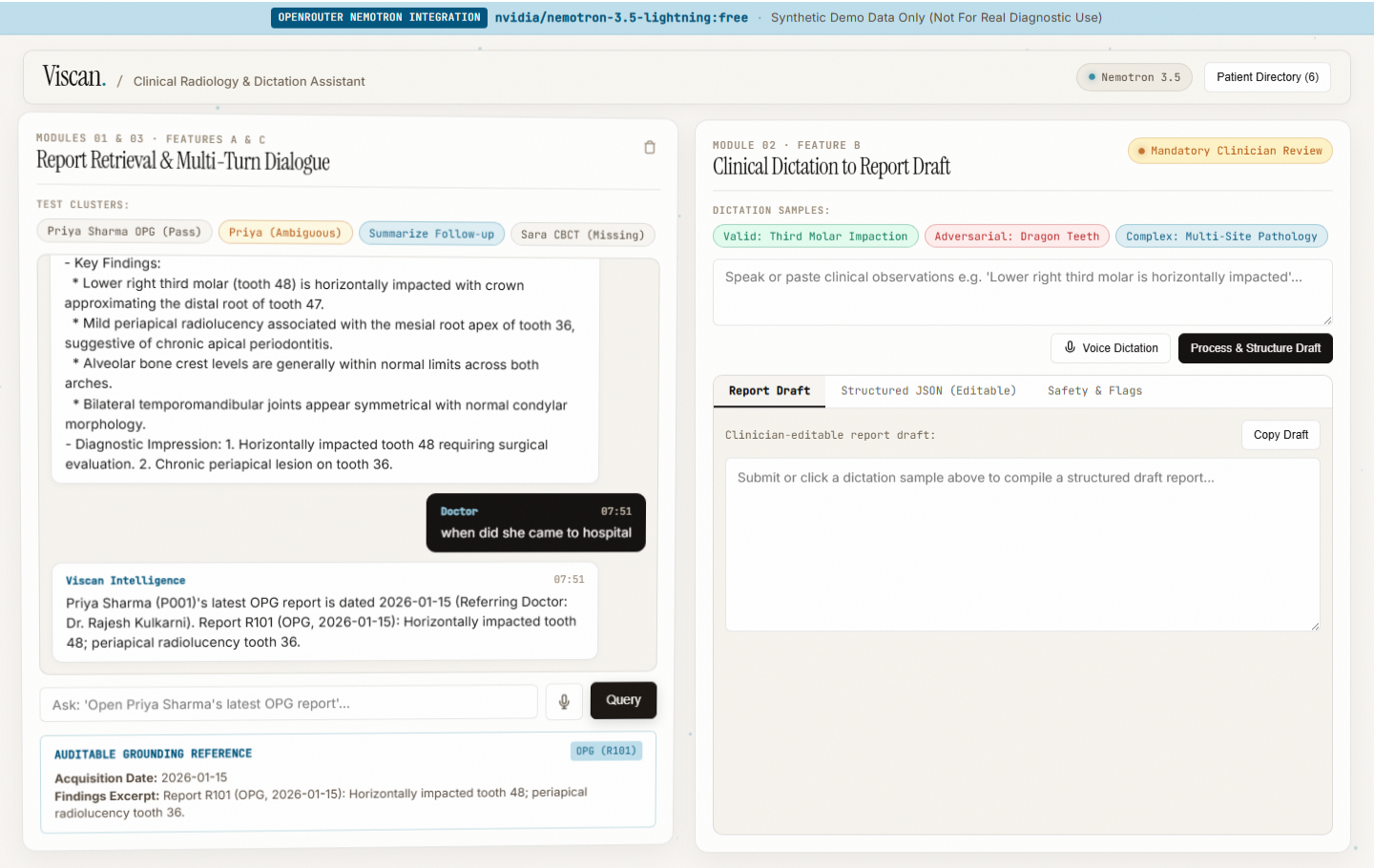
Verification: Response: HTTP 200 | JSON status: success | Context: Priya Sharma (P001) retained from Turn 1

Scenario 03: Temporal Follow-Up Over Visit History (Feature C)

FEATURE C · VISIT HISTORY RAG

Query / Input: "when did she came to hospital"

Bug I hit and fixed here: In an earlier test, typing 'when did she came to hospital' failed with *Patient 'when did she came to hospital' not found*. The name extractor did not strip question prefixes, so it treated the whole sentence as a patient name. I updated the router in app/llm.py to check for temporal keywords (when, visit, hospital) combined with pronouns before name extraction. It now queries the datastore for all reports under P001 and returns her recorded dates: 2026-02-01 (CBCT), 2026-01-15 (OPG), and 2025-06-10 (OPG).



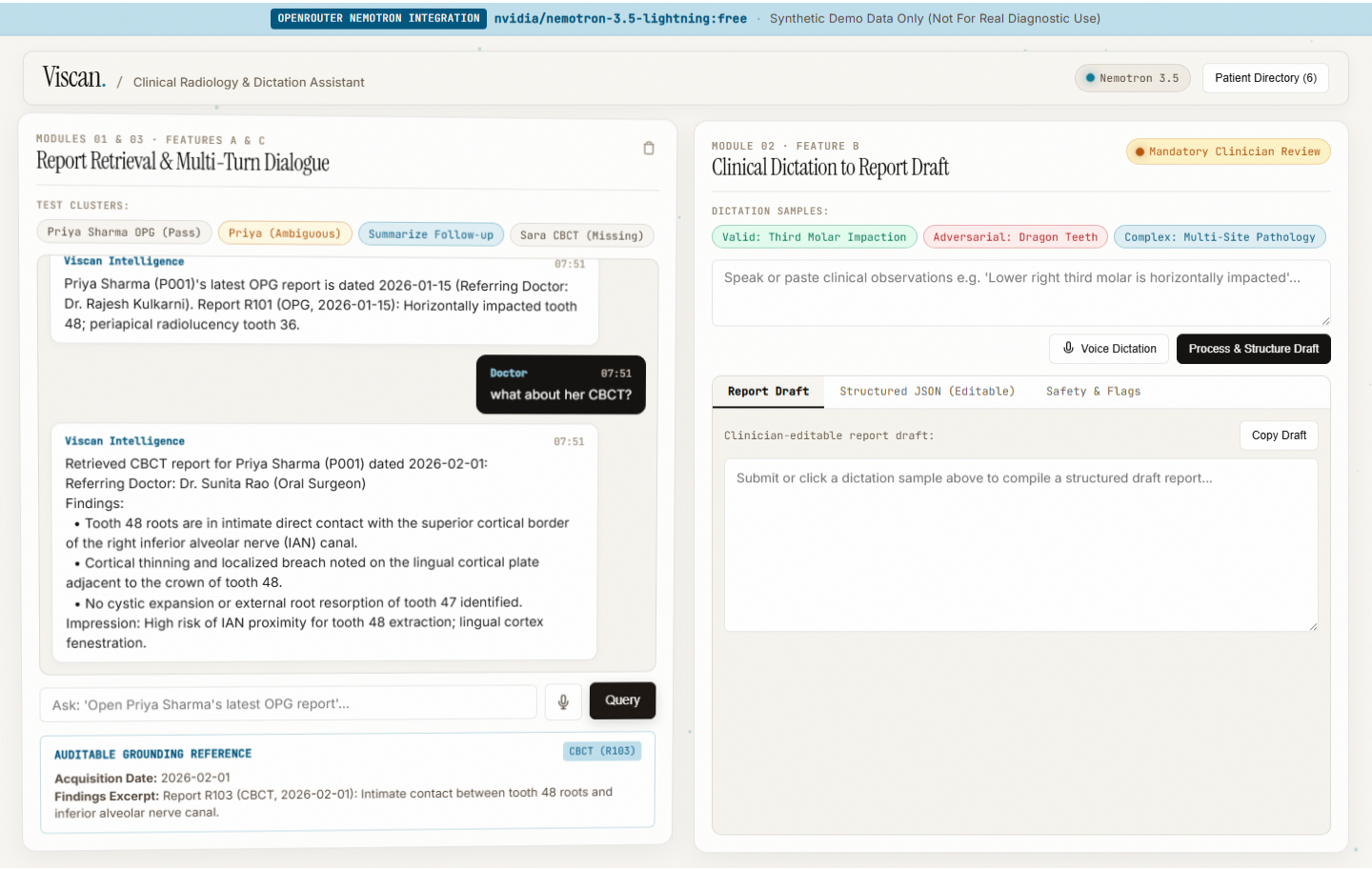
Verification: Response: HTTP 200 | JSON status: success | Resolved: 3 historical visits returned for P001

Scenario 04: In-Context Modality Switch (Feature C)

FEATURE C · MODALITY PIVOT

Query / Input: "what about her CBCT?"

Input: Doctor asks for 'her CBCT' in the active session.
Execution: The router detects the modality switch keyword 'CBCT' while keeping Priya Sharma (P001) as the active patient. It queries get_latest_report('P001', 'CBCT') and returns R103 (2026-02-01), noting tooth 48's close contact with the inferior alveolar nerve canal. session.last_report_id is updated to R103.



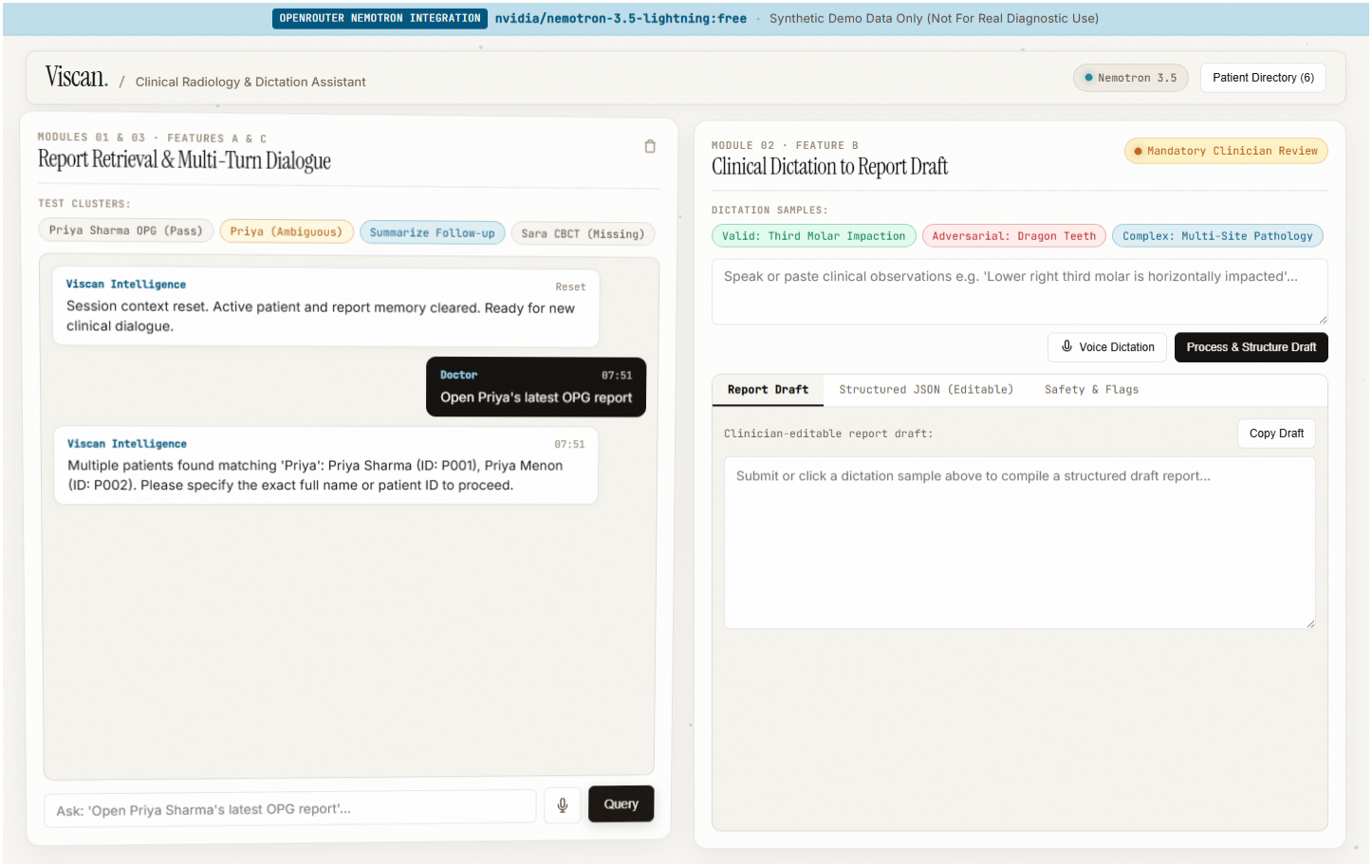
Verification: Response: HTTP 200 | JSON status: success | Active patient kept; report pivoted to R103 (CBCT)

Scenario 05: Ambiguity Guardrail (Feature A)

FEATURE A · AMBIGUITY HANDLING

Query / Input: "Open Priya's latest OPG report"

Input: User provides only first name 'Priya'.
Design choice: The datastore has two patients matching 'Priya': Priya Sharma (P001) and Priya Menon (P002). The system does not select a default or guess based on recency. It halts, returns application status ambiguous, and prints both candidate names and IDs. The user must provide the full name or ID to proceed.



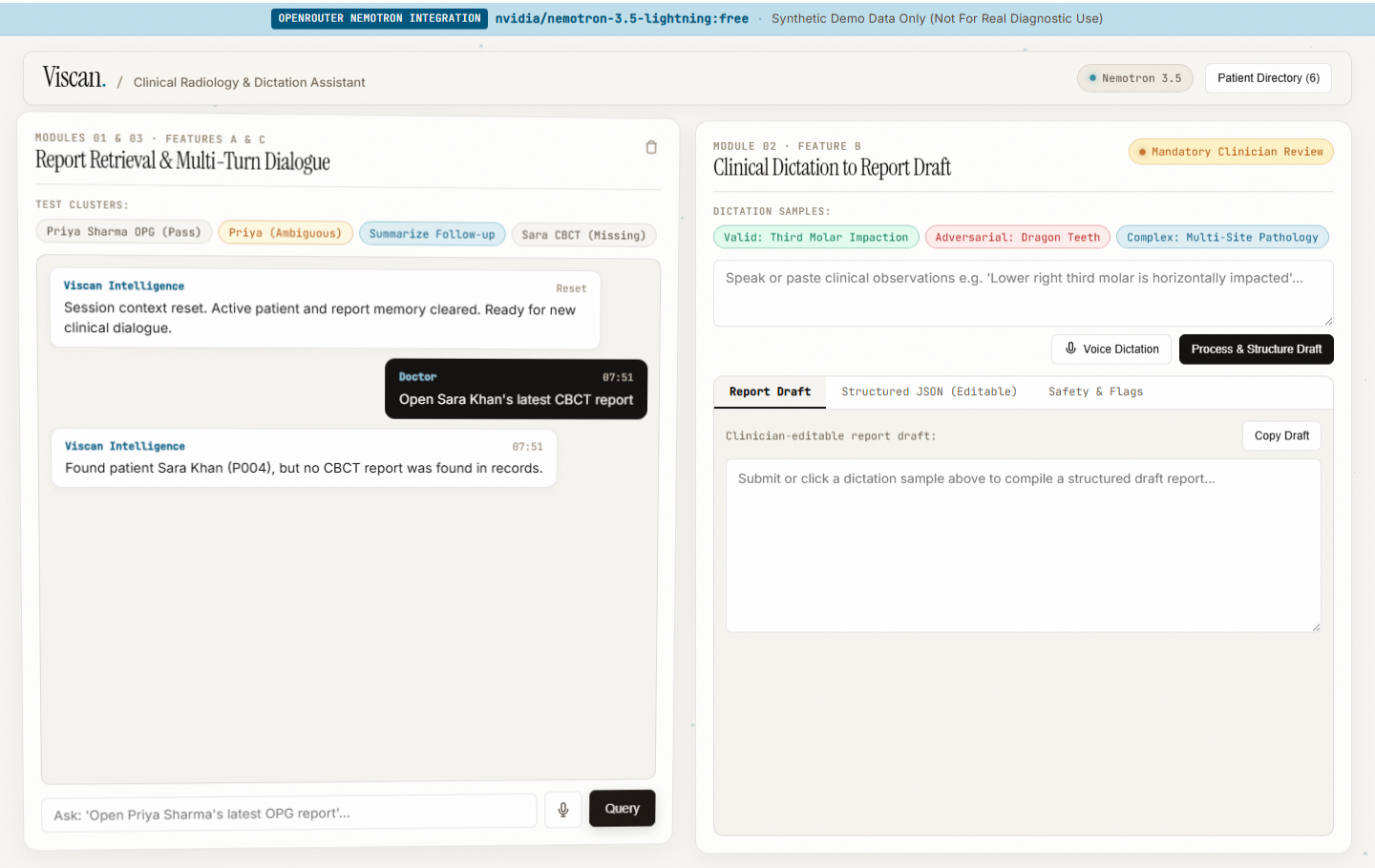
Verification: Response: HTTP 200 | JSON status: ambiguous | Candidates returned: P001, P002 (no record opened)

Scenario 06: Missing Modality Handling (Feature A)

FEATURE A · MISSING RECORD

Query / Input: "Open Sara Khan's latest CBCT report"

Input: User requests a CBCT for Sara Khan (P004).
Execution: Sara Khan exists in patients.json, but only has an IOPA periapical record. Instead of generating mock findings or failing silently, the app returns: *'Found patient Sara Khan (P004), but no CBCT report was found in records.'*



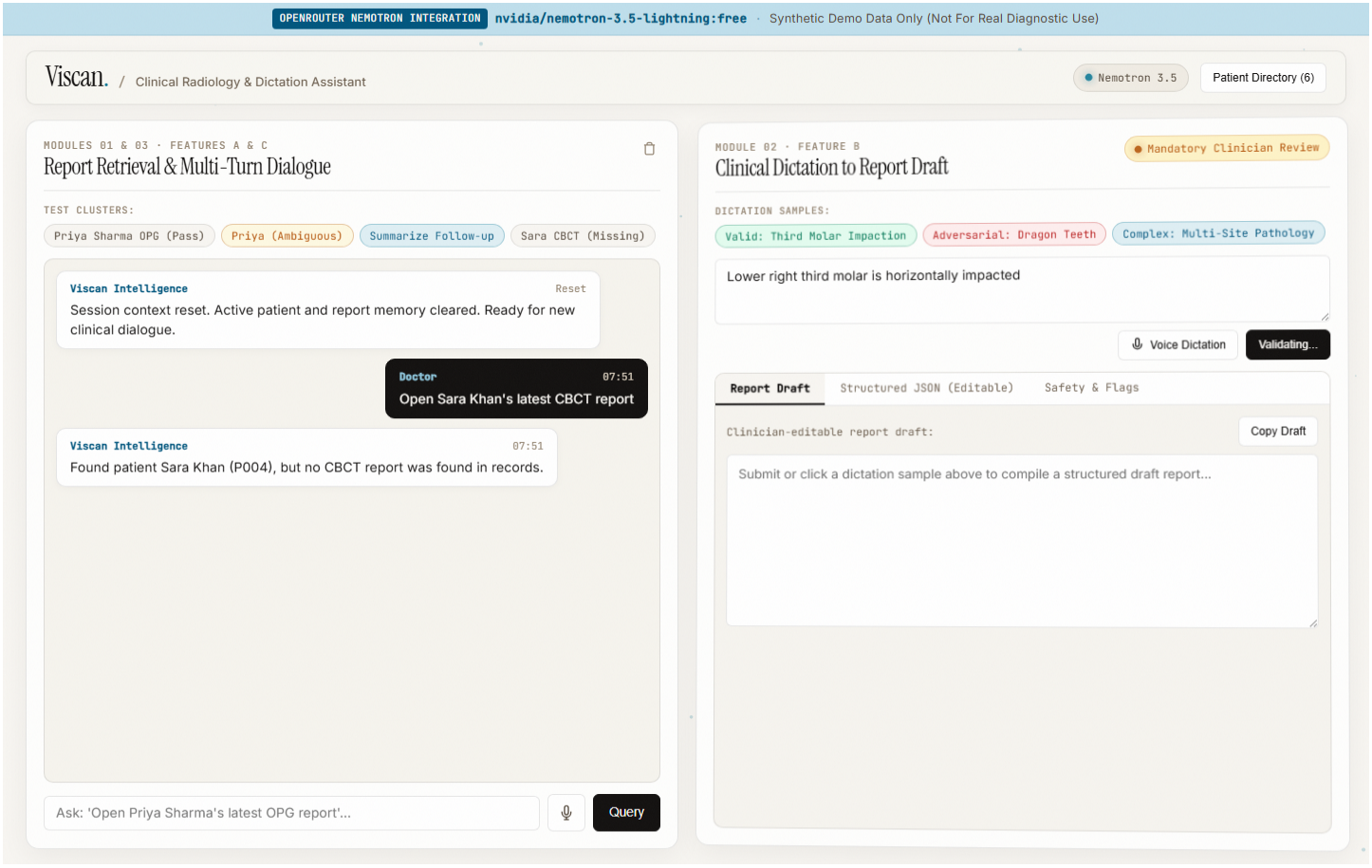
Verification: Response: HTTP 200 | JSON status: not_found | Patient found, requested modality missing

Scenario 07: Clinical Dictation to Draft (Feature B)

FEATURE B · DICTATION STRUCTURING

Query / Input: "Lower right third molar is horizontally impacted"

Input: Doctor dictates clinical observation into the Right Panel.
Execution: The parser isolates anatomical site ('Lower right third molar') and observation ('horizontally impacted'). app/validators.py validates 'Lower right third molar' against FDI tooth 48 and confirms 'horizontally impacted' in the dental pathology dictionary. A draft report is compiled with the status line requires_review: true.



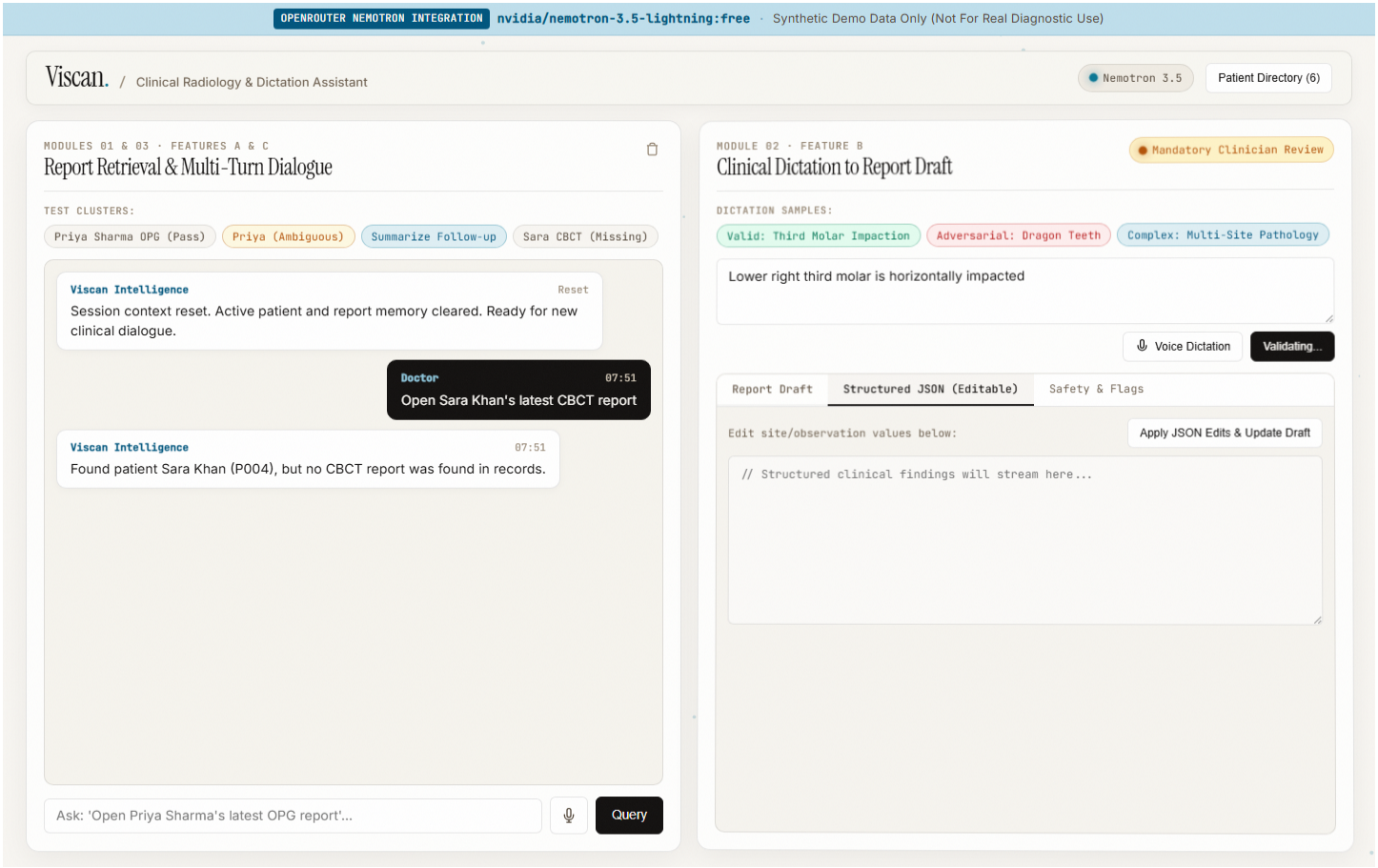
Verification: Response: HTTP 200 | is_supported: true | confidence: 0.95 | requires_review: true

Scenario 08: Structured JSON View & Manual Edits (Feature B)

FEATURE B · EDITABLE JSON PAYLOAD

Query / Input: "Inspection of Structured JSON Tab"

Human-in-the-loop workflow: Selecting the 'Structured JSON' tab exposes the raw parsed array of {anatomical_site, observation, confidence, is_supported} objects. The textarea is editable. A clinician can correct a misheard tooth number or observation and click 'Apply Edits to Draft' to update the generated text before exporting.



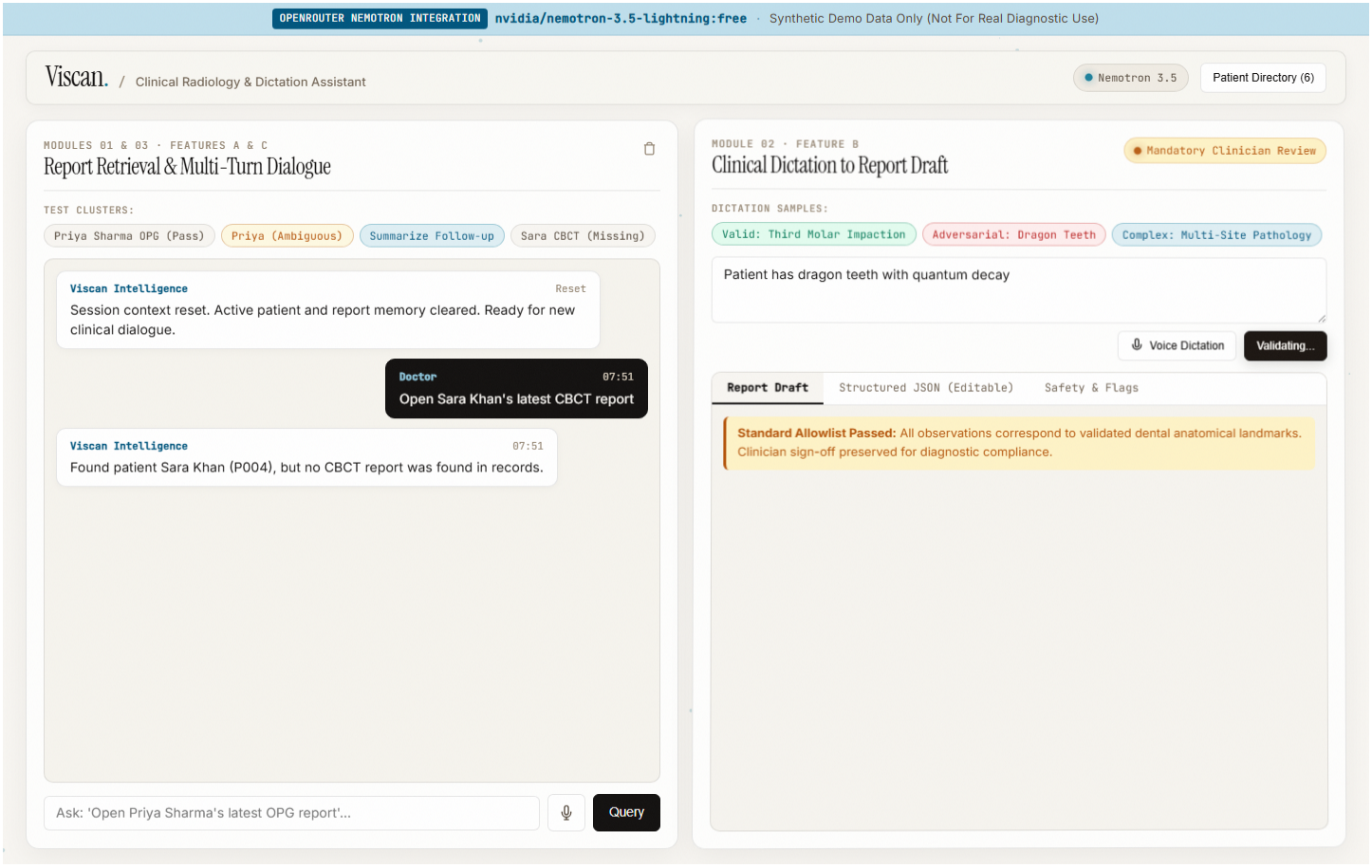
Verification: State: Frontend JSON editor active | Allows clinician overrides before saving

Scenario 09: Rejecting Fictitious Clinical Claims (Feature B)

FEATURE B · DICTIONARY VALIDATION

Query / Input: "Patient has dragon teeth with quantum decay"

Adversarial test: Non-dental phrases were passed to test hallucination resistance.
Execution: The terms 'dragon teeth' and 'quantum decay' fail allowlist validation in app/validators.py. The validator sets is_supported: false, drops confidence to 0.30, adds both terms to unsupported_claims, and inserts warning banners into the draft text. The 'Safety & Flags' tab highlights both unverified entities.



Verification: Response: HTTP 200 | is_supported: false | confidence: 0.30 | flagged in unsupported_claims

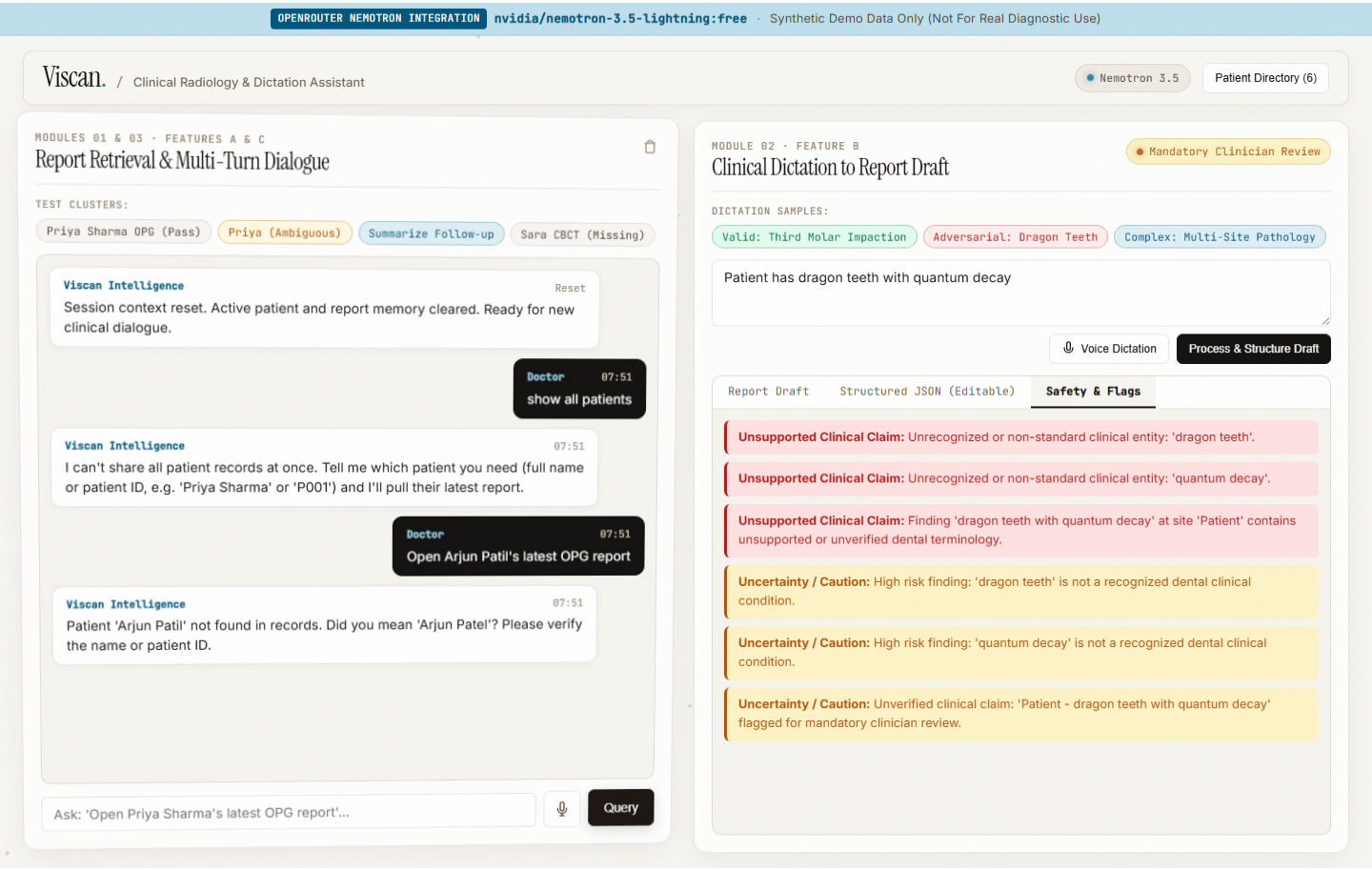
Scenario 10: Bulk Dump Guard & Typo Suggestion

SAFETY · BULK DUMP & SPELL CHECK

Query / Input: "show all patients & Open Arjun Patil's latest OPG report"

Test 1 (Bulk data dump): Querying 'show all patients' matches a regex filter in app/llm.py. The endpoint returns guardrail_blocked, refusing to print the full patient directory.

Test 2 (Typo check): Querying 'Arjun Patil' checks known names with difflib.get_close_matches. It does not open Arjun Patel's record automatically; it returns *Patient 'Arjun Patil' not found. Did you mean 'Arjun Patel'?*



Verification: Bulk query blocked | Typo halted with suggestion (no automatic false-positive opening)